



Pandas for Data Science

Day 3 — Lecture Notes & Practical Tasks

Series · DataFrame · Indexing · Filtering · Missing Values

1. Introduction to Pandas

Pandas is the primary Python library for structured/tabular data. Built on NumPy, it adds labelled axes, mixed data types, and a huge toolset for data cleaning, reshaping, and analysis.

1.1 Why Pandas?

- Python lists and NumPy lack column labels and mixed-type support
- Pandas Series and DataFrame mirror spreadsheets but with code power
- Reads CSV, Excel, JSON, SQL, Parquet — all with a single function call
- Integrates directly with Matplotlib, Seaborn, and Scikit-learn
- Handles real-world messy data: nulls, mixed types, duplicates, dates

1.2 Import Convention

```
import pandas as pd
import numpy as np

print(pd.__version__)

# Quick DataFrame from a dict
data = {'Name': ['Alice', 'Bob', 'Carol'], 'Age': [25, 30, 27],
        'Score': [88.5, 91.0, 76.3]}
df = pd.DataFrame(data)
print(df)
print(df.dtypes)
```

2. Pandas Series

A Series is a 1-dimensional labelled array. Think of it as a single column with an index.

2.1 Creating Series

```
import pandas as pd

# From list (default integer index)
s = pd.Series([10, 20, 30, 40])
print(s)

# Custom index
marks = pd.Series([88, 75, 92, 60], index=['Math', 'Sci', 'Eng', 'Hist'])
print(marks['Math'])      # 88
print(marks[marks > 80])  # filter

# From dict
d = pd.Series({'a':1, 'b':2, 'c':3})

# Arithmetic aligns by index
s1 = pd.Series([1, 2, 3], index=['a', 'b', 'c'])
s2 = pd.Series([10, 20, 30], index=['b', 'c', 'd'])
print(s1 + s2)  # a:NaN b:22 c:33 d:NaN

# Useful attributes
print(marks.values)  # NumPy array
print(marks.index)   # Index object
print(marks.dtype)
```

2.2 Tasks — Series

Task 1: Create a Series of 7 temperatures (float) with days of the week as index. Find the hottest and coldest day using `.idxmax()` and `.idxmin()`.

Task 2: Create two Series with overlapping indices. Add them — explain where NaN appears and how to fill it using `.add(fill_value=0)`.

Task 3: From the Series [45, 82, 67, 55, 91, 38, 74], filter all values above the mean. Print both the mask and the filtered values.

3. DataFrame — Creation & Inspection

3.1 Creating DataFrames

```
import pandas as pd, numpy as np

# From dict of lists
df =
pd.DataFrame({'Name': ['Alice', 'Bob'], 'Age': [25, 30], 'Score': [88.5, 91.0]})

# From NumPy array
arr = np.random.randint(50, 100, (5, 3))
```

```
df2 = pd.DataFrame(arr, columns=['Math', 'Sci', 'Eng'])

# From list of dicts
rows = [{'id':1, 'val':'a'}, {'id':2, 'val':'b'}]
df3 = pd.DataFrame(rows)

# From CSV (most common in practice)
# df = pd.read_csv('data.csv')
# df = pd.read_excel('data.xlsx', sheet_name='Sheet1')
```

3.2 Inspection Functions

```
import pandas as pd, seaborn as sns

df = sns.load_dataset('titanic')

print(df.shape)           # (891, 15)
print(df.ndim)            # 2
print(df.columns.tolist()) # column names
print(df.dtypes)          # dtype of each column
print(df.info())          # non-null counts + dtypes
print(df.describe())      # numeric summary
print(df.describe(include='object')) # string column summary
print(df.head(5))         # first 5 rows
print(df.tail(3))         # last 3 rows
print(df['survived'].value_counts()) # frequency count
print(df['pclass'].nunique()) # unique count
```

3.3 Tasks — DataFrame Basics

Task 1: Create a 5-student DataFrame with columns: Name, Age, Math, Science, English. Calculate and add a Total column.

Task 2: Load the Titanic dataset (seaborn). Print shape, dtypes, and count of missing values per column.

Task 3: Use describe() to find: which numeric column has the highest standard deviation? Which has the most skewed distribution?

Task 4: Write a function df_summary(df) that prints: shape, column names, dtypes, null count per column, and first 3 rows.

4. Indexing & Selection

4.1 Column Selection

```
import pandas as pd, seaborn as sns

df = sns.load_dataset('titanic')

# Single column → Series
```

```
ages = df['age']
print(type(ages))          # <class 'pandas.core.series.Series'>

# Multiple columns → DataFrame
subset = df[['age', 'fare', 'survived']]
print(subset.head())
```

4.2 .loc and .iloc

```
import pandas as pd, numpy as np

df = pd.DataFrame(np.arange(12).reshape(4,3),
                  index=['a','b','c','d'],
                  columns=['X','Y','Z'])

# .loc — label based
print(df.loc['b', 'Y'])      # 4
print(df.loc['a':'c', 'X':'Y']) # slice

# .iloc — integer based
print(df.iloc[1, 1])         # 4
print(df.iloc[0:3, 0:2])     # slice

# at / iat — fast single value access
print(df.at['b','Y'])        # 4
print(df.iat[1,1])          # 4
```

4.3 Boolean Filtering

```
import pandas as pd, seaborn as sns

df = sns.load_dataset('titanic')

# Single condition
adults = df[df['age'] > 18]

# Multiple — use & |, NOT 'and'/'or'
survived_female = df[(df['survived']==1) & (df['sex']=='female')]

# .query — SQL-like (clean and readable)
rich = df.query('fare > 100 and pclass == 1')

# isin — match list of values
top_classes = df[df['pclass'].isin([1,2])]

# between
mid_age = df[df['age'].between(20,40)]

# ~ (not / inverse)
not_male = df[~(df['sex']=='male')]
```

4.4 Tasks — Indexing & Filtering

Task 1: From Titanic, use .loc to extract passengers in rows 10–20 with only age, sex, fare, survived columns.

Task 2: Use .iloc to extract the bottom-right 5×3 sub-matrix of the Titanic DataFrame.

Task 3: Filter: female passengers in 1st class who survived. Print count and average fare.

Task 4: Use .query() to find passengers with fare > 200 OR (pclass==1 AND age < 10). How many are there?

Task 5: From a 6×4 student marks DataFrame, select students where ALL 4 subject marks exceed 60.

5. Missing Values

5.1 Detecting & Handling

```
import pandas as pd, seaborn as sns, numpy as np

df = sns.load_dataset('titanic')

# Detect
print(df.isnull().sum())
print(df.isnull().sum() / len(df) * 100) # % missing

# Drop rows with ANY null
clean = df.dropna()

# Drop rows null in specific column
df2 = df.dropna(subset=['age', 'embarked'])

# Fill with statistics
df['age'] = df['age'].fillna(df['age'].median())
df['embarked'] = df['embarked'].fillna(df['embarked'].mode()[0])

# Fill with constant
df['deck'] = df['deck'].fillna('Unknown')

# Time series fill
ts = pd.DataFrame({'val': [1, np.nan, np.nan, 4]})
print(ts.fillna(method='ffill')) # forward fill
print(ts.fillna(method='bfill')) # backward fill

# interpolate
print(ts.interpolate()) # linear interpolation
```

5.2 Tasks — Missing Values

Task 1: On the Titanic dataset: (a) find % missing per column, (b) drop columns with >50% missing, (c) fill Age with median grouped by Pclass.

Task 2: Create a DataFrame with intentional NaN values. Compare dropna(), fillna(0), fillna(mean), and interpolate() — show the result of each.

Task 3: Use df.fillna(df.mean()) to fill all numeric nulls at once. Then use df.fillna(df.mode().iloc[0]) for non-numeric nulls.

6. Column Operations

```
import pandas as pd, seaborn as sns

df = sns.load_dataset('titanic').copy()

# Add computed column
df['family_size'] = df['sibsp'] + df['parch'] + 1

# Conditional column
df['alone'] = (df['family_size'] == 1).astype(int)

# apply with lambda
df['age_group'] = df['age'].apply(
    lambda x: 'Child' if x<18 else 'Senior' if x>=60 else 'Adult')

# Drop columns
df.drop(columns=['deck', 'embark_town'], inplace=True)

# Rename
df.rename(columns={'pclass': 'Pclass', 'survived': 'Survived'},
          inplace=True)

# Change dtype
df['age'] = df['age'].astype(int)

print(df[['family_size', 'alone', 'age_group']].head())
```

6.1 Tasks — Column Operations

Task 1: On Titanic: add FamilySize, IsAlone (1/0), and AgeGroup ('Child'/'<18', 'Adult'/'18-60', 'Senior'/'>60').

Task 2: Rename all Titanic columns to TitleCase. Then reset them back to lowercase using a single rename call.

Task 3: Convert the 'fare' column from float64 to int. What information is lost? How would you round instead of truncate?

Task 4: Drop all columns that have any missing values. Print shape before and after.

7. Comprehensive Challenges — Day 3

Solve each challenge using only Pandas (+ NumPy where needed). Write clean, commented code.

7.1 Student Report Card

Create a 10-student DataFrame with Name, Math, Science, English, History. (a) Add Total and Average columns. (b) Add Grade: A(≥ 80), B(≥ 65), C(≥ 50), F(< 50). (c) Filter: students with Average > 70 AND no subject below 50. (d) Sort by Average descending.

7.2 Titanic Deep Dive

Load Titanic via seaborn. (a) Fill all missing values appropriately. (b) Add FamilySize and IsAlone. (c) Filter: children (< 18) who survived in 3rd class. (d) What % of women in 1st class survived? (e) Export cleaned DataFrame to titanic_clean.csv.

7.3 Sales Data Wrangling

Create a 20-row sales DataFrame: Date, Product, Region, Units, Price. (a) Add Revenue = Units $\times$ Price. (b) Filter: Revenue > 5000 . (c) Find the top 3 products by total revenue. (d) Check for any missing values and handle them. (e) Export to sales_clean.csv.